



Statischer Sicherheits- und Coding-Guidelines-Scanner

für Oracle PL/SQL und PostgreSQL · Version 2.20.1

Entwicklung · Manuelle Verwendung · CI/CD-Integration

Agenda

- | | | | | | |
|----------|--|----------|----------|--|----------|
| 1 | Einleitung & Überblick
Was ist ACI, Hauptmerkmale, Einschränkungen | S. 3–6 | 5 | Reportformate
Konsole, JSON, HTML, SARIF | S. 32–36 |
| 2 | Architektur & Funktionsweise
Pipeline, IR, Module, Dialekte | S. 7–13 | 6 | CI/CD-Integration
Gate, Waiver, Regelintegrität | S. 37–46 |
| 3 | Prüfungen
Sicherheits-Checks, MITRE ATT&CK, Coding Guidelines | S. 14–22 | 7 | Betrieb, Debugging & Grenzen
Fehlersuche, False Positives, Roadmap | S. 47–52 |
| 4 | Entwicklung & manuelle Verwendung
Setup, CLI, Regeldateien | S. 23–31 | | | |

ACI in 60 Sekunden

2

Prüfgruppen

Sicherheit & Coding Guidelines

~40

Aktive Regeln

5 Security + ~35 Guidelines

2

SQL-Dialekte

Oracle · PostgreSQL

4

Reportformate

Console · JSON · HTML · SARIF

- ✓ Statische Analyse — kein Datenbankzugriff, keine Laufzeitabhängigkeiten
- ✓ JSON-Regeldateien — Erweiterung ohne Code-Änderung
- ! Heuristisch — ersetzt kein manuelles Code-Review

- ✓ Reine Python-Standardbibliothek — keine externen Pakete nötig
- ✓ CI/CD-fähig — Exit-Codes, Waiver, Ruleset-Hash
- ! Empfehlung für CI-Gate: --fail-on high (nicht critical)

Was ist ACI?

ACI (Automated Code Inspection) ist ein statisches Analyse-Werkzeug, das Oracle PL/SQL und PostgreSQL PL/pgSQL prüft — ohne den Code auszuführen.

Sicherheit

- SQL-Injection
- Unerwünschte Packages
- Obfuskiertes Code
- DDL im Code
- Namenskonventionen

Skala: Warning · High · Critical

MITRE ATT&CK

- Persistence-Indikatoren
- Defense Evasion
- Credential Access
- Lateral Movement
- C2 / Exfiltration

Oracle + PostgreSQL · ergänzt Sicherheit

Coding Guidelines

- Oracle: Trivadis v4.4, ~35 Regeln
- PostgreSQL: ACI-eigen, 16 Regeln
- Stil, Wartbarkeit, Zuverlässigkeit
- IDs: G-NNNN / PG-NNNN

Skala: Info · Minor · Major · Critical · Blocker

ACI Historie

Von Version 0.01 zum heutigen ACI 2.20.1 — die wichtigsten qualitativen Sprünge

Version 0.01 — Prototyp (2025)

```
alexanderkornbrust@Macbook-Pro-M5 20250728 % ./aci9.sh

=====
ACI - Automated Code Inspection 0.01
=====

Kommentar: Grant DBA nicht erlaubt
Rule001 (Prio 1) - ./test.sql:2 - grant dba to public;
Kommentar: Grant DBA nicht erlaubt
Rule001 (Prio 1) - ./test.sql:3 - grant dba to public;
Kommentar: Verwendung von TRANSLATE ist nicht erlaubt
Rule003 (Prio 2) - ./test.sql:6 - select translate(value,'ABC','DEF') from table;
Kommentar: Verwendung von DBMS_OBFUSCATION_TOOLKIT ist nicht erlaubt
Rule004 (Prio 1) - ./procs.sql:80 - dbms_obfuscation_toolkit.DESDecrypt(input => raw_input,
Kommentar: Verwendung von DBMS_OBFUSCATION_TOOLKIT ist nicht erlaubt
Rule004 (Prio 1) - ./procs.sql:88 - dbms_obfuscation_toolkit.DESDecrypt(input => encrypted_raw,
Kommentar: Verwendung von DBMS_CRYPTO ist nicht erlaubt
Rule005 (Prio 1) - ./procs.sql:118 - v_hash PLS_INTEGER := DBMS_CRYPTO.HMAC_SH1; -- Hashingalgorithmus SHA-1 mit Passwort
Kommentar: Verwendung von DBMS_CRYPTO ist nicht erlaubt
Rule005 (Prio 1) - ./procs.sql:123 - v_hashwert := DBMS_CRYPTO.MAC(
Kommentar: Verwendung von DBMS_CRYPTO ist nicht erlaubt
Rule005 (Prio 1) - ./procs.sql:124 - src => UTL_I18N.STRING_TO_RAW(eingabe, 'AL32UTF8'), -- DBMS_CRYPTO erfordert Datentyp RAW
Kommentar: Verwendung von DBMS_CRYPTO ist nicht erlaubt
Rule005 (Prio 1) - ./procs.sql:142 - l_encrypted_raw := DBMS_CRYPTO.encrypt(src => l_ccn_raw,
Kommentar: Verwendung von DBMS_CRYPTO ist nicht erlaubt
Rule005 (Prio 1) - ./procs.sql:143 - typ => DBMS_CRYPTO.des_cbc_pkcs5,
Kommentar: Verwendung von DBMS_CRYPTO ist nicht erlaubt
Rule005 (Prio 1) - ./procs.sql:147 - l_decrypted_raw := DBMS_CRYPTO.decrypt(src => l_encrypted_raw,
Kommentar: Verwendung von DBMS_CRYPTO ist nicht erlaubt
Rule005 (Prio 1) - ./procs.sql:148 - typ => DBMS_CRYPTO.des_cbc_pkcs5,
Kommentar: Erstellung von MATERIALIZED VIEW ist nicht erlaubt
Rule006 (Prio 3) - ./procs.sql:1 - CREATE MATERIALIZED VIEW PRODUCT_CATEGORY_AVERAGE AS

FINDINGS

Zusammenfassung der Findings:
Macbook-Pro-M5.local - 20250728 - 2026-05-25 12:49:15
Prio 1: 11
Prio 2: 1
Prio 3: 1
```

Version 2.20.1 — heute

CRITICAL

plsq
l_so
urce
_cod
e_an
alys
is/
6.we
bsv
n/pr
oced
ure
s.sq
l

2904:3

Mögliche SQL-Injection: EXECUTE IMMEDIATE führt dynamisches SQL aus - die Konkatenation enthält ungeprüfte/unvalidierte Werte. Der SQL-String wird in Zeile 2902 gebildet.

```
2903      v_NoC := v_NoC + 1;
2904      EXECUTE IMMEDIATE v_Cmd;
2905  END LOOP;
```

↩ Taint-Quelle: Zuweisung des dynamischen SQL-Strings — Zeile 2902

```
2899
2900 --      Statistics
2901 FOR s IN Cur_Stat LOOP
2902      v_Cmd := 'ANALYZE TABLE ' || s.Table_Name || ' COMPUTE STATISTICS';
2903      v_NoC := v_NoC + 1;
2904      EXECUTE IMMEDIATE v_Cmd;
2905  END LOOP;
```

Empfehlung: Werte über Bindevariablen (USING) statt per String-Konkatenation übergeben. Objektnamen mit DBMS_ASSERT (Oracle) bzw. quote_ident()/format() %i (PostgreSQL) absichern.

Vom Suchen zum Verstehen • SQL-Injection statt Regex-Alarm • ROT13 → Integritäts-Hash • Produktreife (1057 Tests, 4 Reportformate, CI/CD)

Gleich geblieben: regelbasierte statische Prüfung mit nach Schweregrad sortierten Befunden — und der Name ACI.

Hauptmerkmale

01

Zwei Prüfgruppen

Sicherheit und Coding Guidelines mit getrennten Schweregrad-Skalen

02

5 Sicherheits-Checks

Plus ca. 35 Oracle-Guidelines (Trivadis v4.4) und 16 PostgreSQL-Guidelines (ACI-eigen)

03

2 SQL-Dialekte

Oracle PL/SQL und PostgreSQL PL/pgSQL

04

4 Ausgabeformate

Konsole, JSON, HTML und SARIF für Code-Scanning

05

JSON-Regeldateien

Extern, ohne Code-Änderung erweiterbar

06

CI/CD-fähig

Auswertbare Exit-Codes, Waiver, Ruleset-Hash

Kritische Einschränkungen



ACI ist kein vollständiger SQL-Compiler — es bleibt ein heuristischer statischer Scanner. Für Audit und Freigabe sind die folgenden Grenzen maßgeblich.

False Positives / Negatives möglich

ACI ersetzt kein manuelles Review. Befunde sind Indikatoren, keine Beweise.

Dynamisches SQL nicht vollständig

Cross-Datei-Konkatenation, Templates, Migrations-Generatoren, datengetriebenes SQL bleiben blind.

Regeldateien = Trust Boundary

Wer aci/rules/ manipulieren kann, schwächt den CI/CD-Gate. Ruleset-Hash prüfen.

Critical ≠ ausreichend für Gate

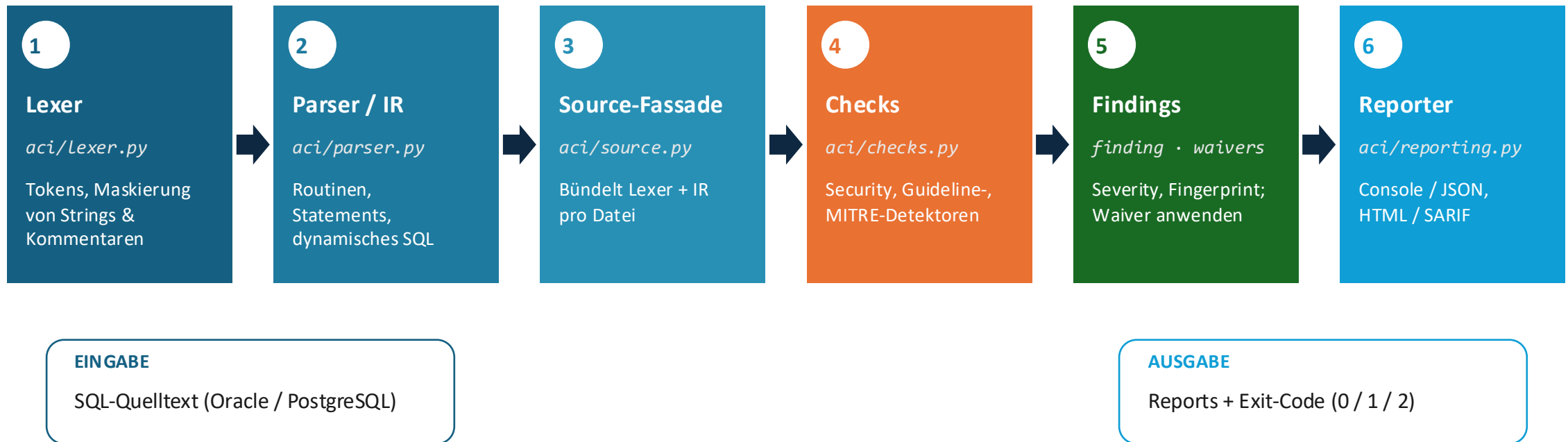
Viele CI-relevante Befunde sind High, nicht Critical. Empfehlung: --fail-on high

PostgreSQL-Policies verstreut

Critical-CI/CD-Policies teils in MITRE/Spezialdetektoren, nicht alle in ddl_in_code.

High-Level-Pipeline

Quelltext durchläuft eine feste Kette von Modulen — von der lexikalischen Analyse bis zum Report.



Orchestrierung: *aci/cli.py:_run()* lädt Regeln, prüft Optionen, verifiziert Regel-Integrität (Ruleset-Hash), scannt, wendet Waiver an und schreibt Reports.

Kernkonzepte

IR

Intermediate Representation

Leichtgewichtige Zwischenform: Routinen, Statements, Zuweisungen, dynamisches SQL. Kein vollständiger SQL-Compiler — bewahrt aber Statement- und Routine-Grenzen sowie die Reihenfolge von Zuweisungen.

MASK

Maskierung

`code_masked` ersetzt Kommentar- und String-Inhalte. So lösen Schlüsselwörter in Kommentaren keine Fehlalarme aus, und Detektoren können gezielt im Code oder im String-Inhalt suchen.

TAINT

Taint-Quelle

Bei SQL-Injection wird neben der Fundstelle (Sink) die Herkunft des dynamischen Strings ausgewiesen — die Zuweisung oder der Routinen-Parameter, der ihn aufbaut.

JSON

Regeln sind Daten

Checks lesen JSON aus `aci/rules/`. Neue Regeln brauchen i.d.R. keinen Code — eine weitere Regel in der passenden Kategoriedatei genügt.

Modulstruktur

Python-Paket — checks/ und reporting/ als Unterpakete; jede Datei kapselt eine Pipeline-Stufe.

ACI/

aci.py	CLI-Einstiegspunkt
aci/	Python-Paket
cli.py	_run() Orchestrierung
finding.py	Finding, Severity
lexer.py	Tokens, Maskierung
parser.py / ir.py	Parser- / IR-Schicht
source.py	Vorverarbeitung
rules.py	Regel-Loader
scanner.py	Scan-Engine
waivers.py	Ausnahmen
integrity.py	Ruleset-Hash
checks/	Unterpaket (Checks)
reporting/	Unterpaket (Reports)
rules/	Regeldateien
oracle.json	Sicherheit (Oracle)
postgresql.json	Sicherheit (PG)
guidelines/oracle/	Coding Guidelines
mitre/oracle/	MITRE-Indikatoren
samples/	Beispieldateien
docs/	Dokumentation

finding.py

Finding, Severity, Fingerprint, Gruppen

parser.py / ir.py

Leichte IR: Routinen, Statements, dyn. SQL

checks/

base · lexical · sql · ddl · detectors · guidelines

reporting/

report · console · json_report · sarif · html

waivers.py

Ausnahmen mit Fingerprint-Bindung

integrity.py

Ruleset-Hash · Trust-Status je Regeldatei

Ein Scan in vier Schritten

Jede Datei durchläuft dieselbe Sequenz — von der Regel-Initialisierung bis zum Report.

1

Regeln laden

Sicherheits-Regeldatei des Dialekts und die Coding-Guideline-Kategoriedateien werden eingelesen und validiert.

2

Vorverarbeitung

Kommentare entfernen, String-Literale erkennen. Zwei Textvarianten entstehen: ohne Kommentare und zusätzlich maskiert. Zeichengenau gleiche Länge — exakte Zeilen/Spalten bleiben erhalten.

3

Checks ausführen

Alle aktiven Checks beider Gruppen laufen über die Quelldatei. Jeder liefert Findings (Regel, Schweregrad, Position, Meldung, ggf. Codeausschnitt und Empfehlung).

4

Bericht erstellen

Findings nach Gruppe getrennt zusammenführen, sortieren und in den gewählten Formaten ausgeben. Waiver werden angewendet — gewaiverte Findings bleiben sichtbar, zählen nicht für --fail-on.

Parser / IR — die Zwischenform

Was die IR leistet

- ✓ Statement-Grenzen bewahren (;-Region)
- ✓ Routine-Grenzen erkennen (END <name>;)
- ✓ Reihenfolge von Zuweisungen erfassen
- ✓ Dynamisches SQL aus Variablen verfolgen
- ✓ 1st-/2nd-order Taint-Klassifikation
- ✓ Shallow Expression-Modell (Calls, Konkatenation)
- ✓ Coarse Control-Block-Marker
- ✓ SELECT/FETCH ... INTO-Quelle erkennen

Was die IR NICHT leistet

- X Kein vollständiger PL/SQL- oder PL/pgSQL-Compiler
- X Keine vollständige Kontrollflussanalyse
- X Keine inter-prozedurale Datenflussanalyse
- X Keine semantische Validierung gegen DB
- X Begrenztes Verständnis generierter SQL-Fragmente
- X Keine Cross-Datei-Datenflussverfolgung
- X False Positives und Negatives bleiben möglich

Taint — Herkunft des dynamischen SQL

Die Analyse ist routine-lokal. Werte über Routinegrenzen gelten konservativ als getaint (lieber False Positive als False Negative).

1st-order · CRITICAL

Dynamisches SQL aus einem ungeprüften Routine-Parameter — Eingabe direkt vom Aufrufer.

```
PROCEDURE delete_user(p_id VARCHAR2) IS
BEGIN
    EXECUTE IMMEDIATE
        'DELETE FROM users WHERE id='' || p_id || ''';
END;
```

Taint-Quelle: Routinen-Header → Sink: EXECUTE IMMEDIATE

2nd-order · CRITICAL

Dynamisches SQL aus einem Tabellen- oder Cursor-Wert — zuvor gespeichert oder gelesen.

```
SELECT col INTO v_cmd FROM config
WHERE key = 'cleanup_sql';

EXECUTE IMMEDIATE v_cmd;
```

Taint-Quelle: SELECT INTO aus Tabelle → Sink: EXECUTE IMMEDIATE

Oracle vs. PostgreSQL

⚠ Der Dialekt wird nicht erraten — ohne `--dialect postgresql` gilt der Oracle-Regelsatz.

ORACLE

Oracle PL/SQL

rules/oracle.json + rules/guidelines/oracle/

Sicherheits-Checks	✓ alle 5
MITRE ATT&CK-Indikatoren	✓ rules/mitre/oracle/
Coding Guidelines	✓ Trivadis v4.4, ~35 Regeln
Dollar-Quotes	—
EnterpriseDB-Pakete	—

Standarddialekt — volle Abdeckung beider Gruppen.

POSTGRES

PostgreSQL PL/pgSQL

rules/postgresql.json

Sicherheits-Checks	✓ angepasst
MITRE ATT&CK-Indikatoren	✓ PG-spezifisch
Coding Guidelines	✓ ACI-eigen, 16 Regeln
Dollar-Quotes	\$func\$ / \$msg\$ verschachteln
EnterpriseDB-Pakete	✓ UTL_HTTP, DBMS_SQL, ...

PostgreSQL: ACI-eigene Detektoren + 16 Coding Guidelines (PG-NNNN).

Zwei Prüfgruppen, zwei Skalen

Jedes Finding gehört zu genau einer Gruppe und trägt einen Schweregrad aus deren Skala. Im Report werden die Gruppen getrennt dargestellt.

SECURITY

Gruppe Sicherheit

Sicherheitsrelevante Befunde — gate-relevant für CI/CD-Pipelines.

- 5 Kern-Checks (SQL-Injection, Packages, ...)
- MITRE ATT&CK-Indikatoren (nur Oracle)
- PostgreSQL-Spezifika (ALTER SYSTEM, RLS)
- Beide Dialekte verfügbar

Skala: Warning · High · Critical

GUIDELINES

Gruppe Coding Guidelines

Stil, Wartbarkeit und Zuverlässigkeit — beratend, nicht blockierend.

- Trivadis PL/SQL Coding Guidelines v4.4
- ~35 aktive Regeln (regex + builtin)
- Original-IDs (G-NNNN) mit Link

Skala: Info · Minor · Major · Critical · Blocker

Skala: Gruppe Sicherheit

Drei Stufen — bewusst kompakt. Für ein verlässliches Deployment-Gate gilt mindestens --fail-on high.

Warning

Auffälligkeit oder Verstoß gegen gute Praxis — Hinweis ohne unmittelbares Risiko.

High

Ernstzunehmendes Risiko, sollte behoben werden — z.B. dynamisches SQL, ausgehende Netzwerkverbindungen, ALTER SYSTEM.

Critical

Schwerwiegende Lücke mit hohem Missbrauchspotenzial — z.B. 1st-order SQL-Injection, obfuskiertes Code.

Skala: Gruppe Coding Guidelines

Fünfstufige Trivadis-Skala. Beratend — Guideline-Findings blockieren standardmäßig keinen CI/CD-Build.

Info	Hinweis ohne unmittelbare Auswirkung.	1
Minor	Geringfügiger Verstoß (Stil, Lesbarkeit).	2
Major	Deutlicher Verstoß gegen Wartbarkeit oder Zuverlässigkeit.	3
Critical	Schwerwiegender Verstoß — sollte korrigiert werden.	4
Blocker	Verstoß, der die Auslieferung blockieren sollte.	5

Internes Gewicht ermöglicht den Vergleich über beide Skalen (z.B. --fail-on high blockiert sowohl Security-High als auch Guideline-Critical/Blocker).

Die 5 Sicherheits-Checks

Gesteuert über rules/oracle.json bzw. rules/postgresql.json — jeder Check ein- und ausschaltbar.

ID	Check	Prüft auf
ACI-NAME	Namenskonventionen	reservierte Worte als Bezeichner, überlange Namen
ACI-PKG	Unerwünschte Packages	sicherheitskritische Packages/Funktionen/Sprachen
ACI-OBF	Obfuskiertes Code	gewrappter, kodierter oder verschleierter Code
ACI-SQLI	SQL Injection	dynamisches SQL mit String-Konkatenation
ACI-DDL	DDL im Code	GRANT / CREATE / ALTER / DROP im Code

Zusätzlich: MITRE ATT&CK-Indikatoren (Oracle & PostgreSQL) für persistenz-, defense-evasion-, credential-access- und C2-Muster.

MITRE-Angriffsindikatoren — Oracle & PostgreSQL

Code-Muster, die auf konkrete Angriffstechniken hindeuten. Ergänzt die fünf Sicherheits-Checks gezielt um Muster, die diese nicht erfassen.

ORACLE

Oracle MITRE-Indikatoren

12 Taktiken · 66 Regeln

TA0002–TA0112

- Execution (DBMS_MLE, CREATE JAVA SOURCE)
- Persistence (AFTER LOGON, Scheduler-Jobs)
- Defense Evasion (NOAUDIT, DELETE AUD\$)
- Credential Access (SYS.USER\$)
- Lateral Movement (DBMS_FILE_TRANSFER)
- Exfiltration/C2 (JDWP, Relay-URLs)

POSTGRESQL

PostgreSQL MITRE-Indikatoren

10 Taktiken · 68 Regeln

seit ACI 2.6+

- Persistence (Event Trigger, Subscription)
- Privilege Escalation (GRANT PUBLIC, Sys-Rollen)
- Defense Evasion (RLS aus, ALTER SYSTEM)
- Lateral Movement (FDW, dblink)
- Collection (\copy, COPY FROM file)
- Exfiltration (COPY TO file, Replication)

Taktiken im Überblick — Oracle vs. PostgreSQL

Pro Taktik je ein typisches Indikator-Muster. Pflegbar als JSON unter `rules/mitre/<dialect>/`.

Taktik	Oracle	PostgreSQL
Execution TA0002	DBMS_MLE, CREATE JAVA SOURCE	—
Persistence TA0003	AFTER LOGON, Scheduler EXECUTABLE	CREATE EVENT TRIGGER, CREATE SUBSCRIPTION
Privilege Escalation TA0004	AUTHID DEFINER, IDENTIFIED BY VALUES	GRANT ... TO PUBLIC, system role grants
Defense Evasion TA0005	_oracle_script, NOAUDIT, DELETE AUD\$	RLS disable, ALTER SYSTEM, SET ROLE
Credential Access TA0006	SYS.USER\$, DBMS_SYSTEM.GET_ENV	—
Lateral Movement TA0008	DBMS_FILE_TRANSFER	FDW (CREATE SERVER), dblink
Collection TA0009	—	\copy, COPY FROM file, Publication
Exfiltration / C2 TA0010/11	JDWP.CONNECT_TCP, NETWORK_ACL_ADMIN	COPY TO file, Subscription credentials
Impact TA0040	CREATE LIBRARY, ADVANCED_REWRITE	pg_terminate_backend, pg_cancel_backend
Defense Impairment TA0112	SQL_FIREWALL.DISABLE	—

PostgreSQL — spezielle Detektoren

PostgreSQL-/EPAS-eigene Regeln decken Administration, RLS, Audit-Tampering, FDW und Extensions ab — Befunde, die bei normalen Migrations-Skripten kritisch sind.

Administration & Rollen

- ALTER SYSTEM SET (work_mem, security)
- ALTER SYSTEM RESET
- GRANT ... WITH GRANT/ADMIN OPTION
- Vordefinierte Rollen (pg_checkpoint, pg_maintain, pg_create_subscription, ...)
- REASSIGN OWNED · Lesen von pg_authid / pg_shadow

Row Level Security & Audit

- Abschalten von RLS
- Änderung von RLS-Policies
- Deaktivieren von Triggern
- Event-Trigger-Manipulation
- Audit/Logging via ALTER SYSTEM abschalten (edb_audit, log_statement)
- pg_reload_conf nach Audit-Änderung · postgresql.auto.conf-Manipulation

Erweiterungen & Code-Loading

- CREATE EXTENSION dblink (Risiko)
- Riskante Extensions: adminpack, pg_cron, pg_net, plv8, http
- LOAD shared library
- Anlegen/Ersetzen in pg_catalog / public

Daten & SECURITY DEFINER

- SECURITY DEFINER ohne festen search_path
- FDW / User Mappings / Logical Replication
- Server-Datei-Operationen, large objects
- \copy (Client-Datei), COPY FROM '/path'
- SECURITY DEFINER mit privilegierter DDL (Audit-Bypass-Kandidat)

Abdeckung — Oracle & PostgreSQL

ACI prüft Coding Guidelines für beide Dialekte. Oracle nach Trivadis-Standard, PostgreSQL als ACI-eigener Regelsatz.

ORACLE

Trivadis-Abdeckung

~35 aktive Regeln

Trivadis PL/SQL & SQL Coding Guidelines v4.4 · Apache-2.0

- G-NNNN-IDs (Originalsystem von Trivadis)
- Verlinkung auf trivadis.github.io
- Skala: Info · Minor · Major · Critical · Blocker
- Eine JSON-Datei je Trivadis-Kategorie
- Detektoren: Regex oder Builtin
- Regeln mit "needs-parser" sind als inaktiv markiert

POSTGRESQL

ACI-eigener Regelsatz

16 aktive Regeln

ACI-eigene PL/pgSQL-Guidelines · Quelle: ACI (kein Trivadis)

- PG-NNNN-IDs (ACI-eigenes Schema)
- Schwerpunkt: Wartbarkeit, Robustheit, Sicherheit
- Skala: Info · Minor · Major (kein Critical/Blocker)
- Eine JSON-Datei: `rules/guidelines/postgresql/g_postgresql.json`
- SECURITY DEFINER-Hinweise (PG-7010 ff.)
- Ergänzt die Sicherheits-Checks, kein Ersatz

Beispiel-Regeln — beide Dialekte

Auswahl typischer aktiver Regeln je Dialekt. Vollständige Liste über `python3 aci.py --list-checks`.

ORACLE

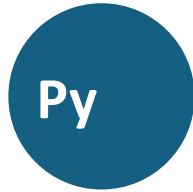
ID	Regel	Level
G-2180	Keine gequoteten Bezeichner	Major
G-2320	Datentyp VARCHAR nie verwenden	Major
G-3145	Kein SELECT * aus Tabelle/View	Major
G-3310	Kein COMMIT in einer Schleife	Critical
G-4310	Keine GOTO-Anweisungen	Major
G-6010	Dynamisches SQL über Variable ausführen	Major
G-7125	CREATE OR REPLACE verwenden	Major
G-9010	Formatmaske bei TO_DATE	Major

POSTGRESQL

ID	Regel	Level
PG-1010	SELECT * vermeiden	Minor
PG-1020	Vergleich mit = NULL statt IS NULL	Major
PG-3310	COMMIT/ROLLBACK in einer Schleife	Major
PG-5010	Leerer WHEN-OTHERS-Handler	Major
PG-7010	SECURITY DEFINER ohne festen search path	Major
PG-7012	SECURITY DEFINER mit dyn. EXECUTE	Major
PG-7020	RAISE NOTICE im Produktionscode	Info
PG-7030	Funktion ohne LANGUAGE-Angabe	Minor

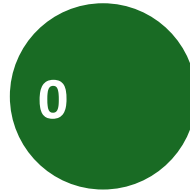
Voraussetzungen

ACI ist bewusst schlank: reine Python-Standardbibliothek, keine externen Pakete erforderlich.



Python 3.9+

Aktuelle Python-Version. `python3 --version` zur Prüfung.



Null Abhängigkeiten

Nur Python-Standardbibliothek. Kein pip install nötig.



Plattformunabhängig

Linux, macOS, Windows. Reines CLI-Tool.

Optionale Entwickler-Werkzeuge

```
pip install .[dev]           # installiert ruff + mypy als Entwickler-Werkzeuge
python3 -m pytest -q         # Testsuite ausführen (keine Laufzeitabhängigkeiten)
python3 -m ruff check .      # Linter (optional)
python3 -m mypy aci          # Typprüfung (optional)
```


Lokales Setup — Schritt für Schritt

Vom Klonen bis zum ersten erfolgreichen Scan in vier Schritten.

1

Python prüfen

```
python3 --version
```

Erwartet: 3.9 oder neuer

2

Verzeichnis betreten

```
cd ACI
```

ACI-Quellverzeichnis

3

Tests laufen lassen

```
python3 -m pytest -q
```

Sanity-Check: alles grün

4

Erster Scan

```
python3 -m aci samples/  
vulnerable_oracle.sql  
-g security
```

Beispieldatei prüfen

Ergebnis: Konsolenausgabe mit Findings aus der Gruppe Sicherheit. Bereit für eigene SQL-Dateien.

Repository-Struktur

Logischer Aufbau des ACI-Pakets — Code links, Regeldateien rechts.

ACI/ – Verzeichnisstruktur

```
ACI/
├── aci.py                Einstiegspunkt (CLI)
├── aci/                 Python-Paket
│   ├── cli.py           _run() Orchestrierung
│   ├── config.py        Standardparameter (aci.ini)
│   ├── finding.py       Finding, Severity
│   ├── lexer.py         Tokens, Maskierung
│   ├── parser.py / ir.py Parser- / IR-Schicht
│   ├── source.py        Vorverarbeitung je Datei
│   ├── rules.py         Regel-Loader
│   ├── scanner.py       Scan-Engine
│   ├── waivers.py       Ausnahmen
│   ├── integrity.py     Ruleset-Hash + Trust
│   ├── checks/          Checks-Unterpaket
│   │   ├── (base · lexical · sqli · ddl · detectors · guidelines)
│   ├── reporting/       Report-Unterpaket
│   │   ├── (report · console · json_report · sarif · html)
│   └── rules/           externe Regeldateien
│       ├── oracle.json  Sicherheit (Oracle)
│       ├── postgresql.json Sicherheit (PostgreSQL)
│       ├── guidelines/oracle/ Trivadis-Kategorien
│       └── mitre/oracle/ MITRE-Taktiken
├── licenses/           LICENSE · Apache-2.0 · NOTICE
├── samples/ · tests/ · docs/
```

Code-Schichten in aci/

Jede Datei = eine Phase der Pipeline (Lexer → Parser/IR → Source → Checks → Reporter), Waiver & Integrität flankieren.

Regeln getrennt vom Code

rules/ liegt außerhalb des Python-Pakets — neue Regeln brauchen keinen Code.

Coding Guidelines

Eine JSON-Datei je Trivadis-Kategorie. Im Report eine gemeinsame Gruppe.

MITRE-Indikatoren

rules/mitre/oracle/ — eine Datei je ATT&CK-Taktik (z.B. ta0003_persistence.json).

Grundlegende Verwendung

Aufrufmuster und die wichtigsten Schalter — alles weitere ist in den Optionstabellen auf den nächsten Folien.

Syntax

```
python3 aci.py <Datei-oder-Verzeichnis> [Optionen]
```

Drei Schalter, die fast immer reichen

-d / --dialect

oracle | postgresql

SQL-Dialekt. Wird nicht erraten — Default: oracle.

-g / --group

all | security | guidelines

Zu prüfende Gruppen. Default: all.

-f / --format

console,json,html,sarif

Reportformate, kommasetrennt kombinierbar.

Vorgabewerte vieler Optionen kommen aus `aci.ini` — ein gesetzter Schalter überschreibt sie. Gebündelte Regelverzeichnisse zeigen auf die mit ACI ausgelieferten Regeldateien.

Optionen — Eingabe & Ausgabe (1/2)

Dialekt, Regelpfade, Reportformate und Schweregrad-Filter.

Option	Beschreibung	Standard
<code>-d, --dialect</code>	oracle oder postgresql	<i>oracle</i>
<code>-g, --group</code>	Prüfgruppen: all, security, guidelines	<i>all</i>
<code>--rules</code>	explizite Sicherheits-Regeldatei	<i>-</i>
<code>--rules-dir</code>	Verzeichnis mit oracle.json / postgresql.json	<i>gebündelt</i>
<code>--guidelines-dir</code>	Basisverzeichnis der Guideline-Kategoriedateien	<i>gebündelt</i>
<code>--mitre-dir</code>	Basisverzeichnis der MITRE-Taktikdateien	<i>gebündelt</i>
<code>-f, --format</code>	console, json, html, sarif (kommagetrennt)	<i>console</i>
<code>-o, --output-dir</code>	Zielverzeichnis für Reports	<i>akt. Verz.</i>
<code>--html-group-by</code>	HTML-Findings nach rule oder file gruppieren	<i>rule</i>
<code>--min-level</code>	nur Findings ab diesem Schweregrad	<i>info</i>

Optionen — Gate, Filter & Debug (2/2)

Gate-, Sicherheits- und Filter-Schalter. Vorgabewerte stammen aus `aci.ini`; ein gesetzter Schalter überschreibt sie.

Option	Beschreibung	Standard
<code>--fail-on</code>	Exit-Code 1, wenn Findings $\geq$ Schweregrad (für CI/CD; Empfehlung: $\geq$ high)	none
<code>--strict-internal-errors</code>	Exit-Code 2, wenn ein Check intern fehlschlägt	–
<code>--waivers</code>	JSON-Datei mit kontrollierten Ausnahmen — gewaiverte Findings zählen nicht für <code>--fail-on</code>	–
<code>--strict-waivers</code>	Exit-Code 2, wenn die Waiver-Datei fehlerhaft ist (sonst nur Warnung)	–
<code>--require-trusted-rules</code>	Exit-Code 2, wenn eine Regeldatei aus benutzerdefiniertem (nicht gebündeltem) Pfad geladen würde	–
<code>--context-lines N</code> / <code>--no-context</code>	Zeilen Quelltext-Kontext pro Finding bzw. ganz weglassen	3
<code>--redact-secrets</code> / <code>--safe-report</code>	Geheimnis-Muster maskieren bzw. <code>--no-context</code> + <code>--redact-secrets</code> bündeln	–
<code>--taint-sources</code> / <code>--no-taint-sources</code>	Taint-Quelle bei SQL-Injection und dynamischen DDL-Findings anzeigen	an
<code>--exclude MUSTER</code> / <code>--max-file-size</code>	Datei-/Verzeichnismuster ausschließen bzw. große Dateien überspringen	–
<code>--debug</code> / <code>--dump-ir</code> / <code>--list-checks</code>	Traceback, IR-JSON oder Liste aktiver Checks ausgeben und beenden	–

Beispiele typischer Aufrufe

Vier typische Szenarien — vom einfachsten Lauf bis zum produktiven CI/CD-Gate.

1 Standard — beide Gruppen

Sicherheit + Coding Guidelines, Konsolenausgabe

```
python3 aci.py samples/vulnerable_oracle.sql
```

2 Nur Sicherheits-Checks

Coding Guidelines überspringen — z.B. für schnellen Sicherheits-Scan

```
python3 aci.py code.sql --group security
```

3 PostgreSQL-Dialekt

PostgreSQL erzwingen (Standard ist Oracle); für PG nur Sicherheit verfügbar

```
python3 aci.py code.sql -d postgresql
```

4 HTML-Report in Verzeichnis

Nur Guidelines, HTML-Ausgabe in reports/-Ordner

```
python3 aci.py code.sql --group guidelines -f html -o reports/
```

Regeldateien — Aufbau einer Regel

Alle Regeln liegen außerhalb des Programmcodes. Eine Regel ist ein JSON-Objekt mit Metadaten und einem Detektor.

```
rules/guidelines/oracle/g3_dml_and_sql.json
```

```
{
  "id": "G-3145",
  "title": "Kein SELECT * aus Tabelle oder View",
  "severity": "Major",
  "characteristics": "Maintainability, Reliability",
  "message": "Verwendung von SELECT *.",
  "recommendation": "Benötigte Spalten explizit aufzählen.",
  "url": "https://trivadis.github.io/.../g-3145/",
  "enabled": true,
  "detector": {
    "type": "regex",
    "target": "code",
    "pattern": "\\bSELECT\\s+\\s+\\s*"
  }
}
```

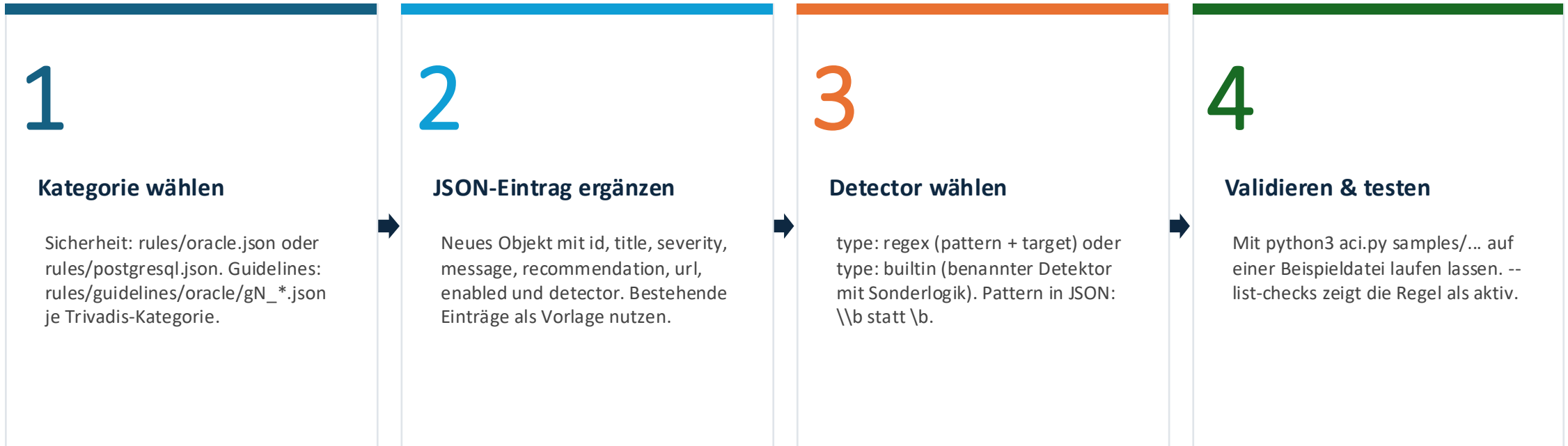
Felder einer Regel

id, title	Eindeutige Regel-ID + Anzeigename
severity	Schweregrad aus der Gruppen-Skala
message	Text, der im Finding erscheint
url	Link zur Regelseite (z.B. Trivadis)
enabled	Regel aktiv? (false = deaktiviert)
detector.type	regex oder builtin (Sonderlogik)
detector.target	code (Quelltext) oder string (Inhalt von String-Literalen)
detector.pattern	Regex; in JSON \\b statt \b

i Auf gültiges JSON achten — Rückwärtsstriche in regulären Ausdrücken JSON-konform verdoppeln (\\b). ACI prüft jede Regeldatei beim Laden und meldet Fehler verständlich.

Eigene Regel hinzufügen — vier Schritte

Neue Regeln brauchen i.d.R. keinen Code — nur einen weiteren Eintrag in der passenden JSON-Datei.



VORTEIL

Regeln sind Daten — kein Python-Code, kein Build-Schritt, kein Restart. Eine neue Regel ist mit einem Pull-Request auf eine JSON-Datei einsatzbereit. Das Team pflegt seinen eigenen Regelstand außerhalb des Programmcodes; der Ruleset-Hash macht den verwendeten Stand jederzeit überprüfbar.

Vier Formate, vier Anwendungsfälle

--format console,json,html,sarif — kommagetrennt kombinierbar. Alle Formate trennen Sicherheit und Coding Guidelines mit eigenen Schweregrad-Zusammenfassungen.



Console

BUILD-LOG

Farbiger Textreport, je Gruppe ein eigener Abschnitt. Mit `--no-color` ohne ANSI-Farben für CI-Logs.



JSON

ARCHIVIERUNG & AUSWERTUNG

Maschinenlesbar. Enthält `scanner_config`, `ruleset.integrity`, ggf. `waivers-Block`, je Finding `group/severity/fingerprint`.



HTML

REVIEW IM BROWSER

Eigenständige HTML-Seite mit Kennzahlen-Karten je Gruppe. `-html-group-by` wählt `rule` oder `file` als Gruppierung.



SARIF

CODE-SCANNING-INTEGRATION

SARIF 2.1.0 für GitHub-/GitLab-/Azure-DevOps-Dashboards. Regelkatalog je `check_id:rule_ref`, Waiver als `suppressions`.

Konsole & JSON im Detail

☰ Konsole

```
$ python3 aci.py samples/vulnerable_oracle.sql
```

```
— Sicherheit —
```

```
[CRIT] ACI-SQLI app.sql:42:7  
1st-order SQL-Injection  
[HIGH] ACI-PKG app.sql:11:5  
UTL_HTTP nicht erlaubt
```

```
— Coding Guidelines —
```

```
[MAJ] G-3145 proc.sql:18  
SELECT * vermeiden
```

```
Regeln bundled (Hash 8f3a...)
```

- Je Gruppe ein eigener Abschnitt mit Schweregrad-Zählern
- Farbig per ANSI; --no-color für CI-Logs
- Zeile »Regeln« zeigt Ruleset-Hash und Vertrauensstatus
- Standardformat, wenn kein --format gesetzt ist

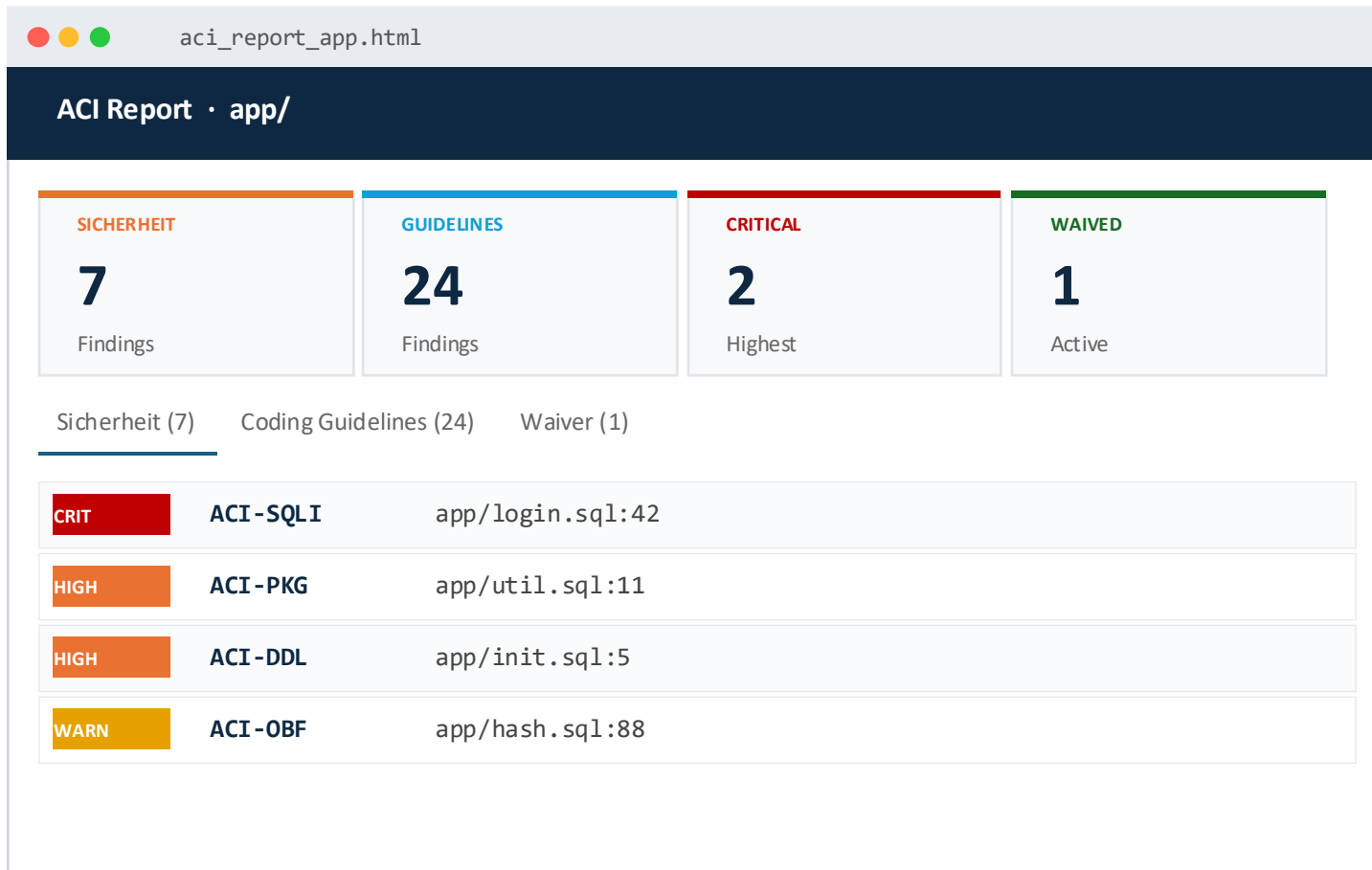
{ } JSON

```
{  
  "scanner_config": { "dialect": "oracle" },  
  "ruleset": {  
    "integrity": "8f3ade...",  
    "trusted": true  
  },  
  "findings": [{  
    "group": "security",  
    "severity": "Critical",  
    "rule": "ACI-SQLI",  
    "fingerprint": "39e7c95257...",  
    "file": "app.sql", "line": 42  
  }],  
  "waivers": { "active": 0, "expired": 0 }  
}
```

- Maschinenlesbar — Eingabe für Dashboards, Skripte
- Findings mit group, severity, fingerprint, url
- ruleset.integrity macht Regelstand prüfbar
- waivers-Block bei genutzter Waiver-Datei

HTML — eigenständige Browser-Seite

Self-contained HTML-Datei mit Kennzahlen-Karten je Gruppe und gruppierten Findings. Ideal für Review-Meetings und Nicht-CI-Konsumenten.



Eigenschaften

Eigenständig

Eine HTML-Datei, keine externen Assets

Gruppierung

--html-group-by rule oder file

KPI-Karten

Je Gruppe Schweregrad-Zähler oben

Drill-Down

Klick auf Regel öffnet Trivadis-Seite (Guidelines)

Quelltext

Inline-Code-Auszüge je Finding (mit --no-context aus)

Waiver

Eigene Sektion, deutlich markiert

SARIF — Integration in Code-Scanning

Static Analysis Results Interchange Format 2.1.0 — der offene Standard, den GitHub, GitLab und Azure DevOps direkt einlesen können.

Spezifika in ACI

- 1 Regelkatalog je Regelreferenz**
tool.driver.rules wird je check_id:rule_ref geführt — nicht nur check_id. So verfolgen Dashboards einzelne Regeln getrennt.
- 2 Suppressions = Waiver**
Gewaiverte Findings werden als suppressions (kind: external) exportiert. Sie bleiben sichtbar und auditierbar.
- 3 Fingerabdruck als partialFingerprints**
Jeder Befund trägt seinen Fingerprint im SARIF-Standardfeld — Bewegungen der Codezeile ändern die Identität nicht.
- 4 Ruleset-Hash als Run-Property**
runs[].properties.aci_ruleset_hash macht den verwendeten Regelstand im Dashboard sichtbar.

aci_report_app.sarif

```
"runs": [{  
  "tool": { "driver": {  
    "name": "ACI",  
    "version": "2.20.1",  
    "rules": [{  
      "id": "ACI-PKG:utl_http",  
      "shortDescription": ...  
    }, {  
      "id": "ACI-OBF:CHR-CHAIN",  
      ...  
    }]  
  }},  
  "properties": {  
    "aci_ruleset_hash": "8f3ade..."  
  },  
  "results": [{  
    "ruleId": "ACI-SQLI:dyn_param",  
    "partialFingerprints": {  
      "aci/v1": "39e7c95257..."  
    },  
    "suppressions": []  
  }]  
}]
```

Reportpfade & Vertraulichkeit

Reports können Dateipfade, Scan-Metadaten und Quelltext-Kontext enthalten — behandle sie als interne Artefakte, sofern nicht bereinigt.



Was Reports preisgeben können

- **Absolute Pfade**
Repository-Strukturen, Workspace-Namen, CI-Runner-Pfade — per `--redact-paths` anonymisierbar.
- **Quelltext-Kontext**
Standardmäßig 3 Zeilen vor und nach jedem Fund.
- **Erkannte Geheimnisse**
Passwörter, API-Keys, Connection-Strings im Codeauszug.
- **Scan-Metadaten**
Hostnamen, Benutzernamen, Toolversionen.

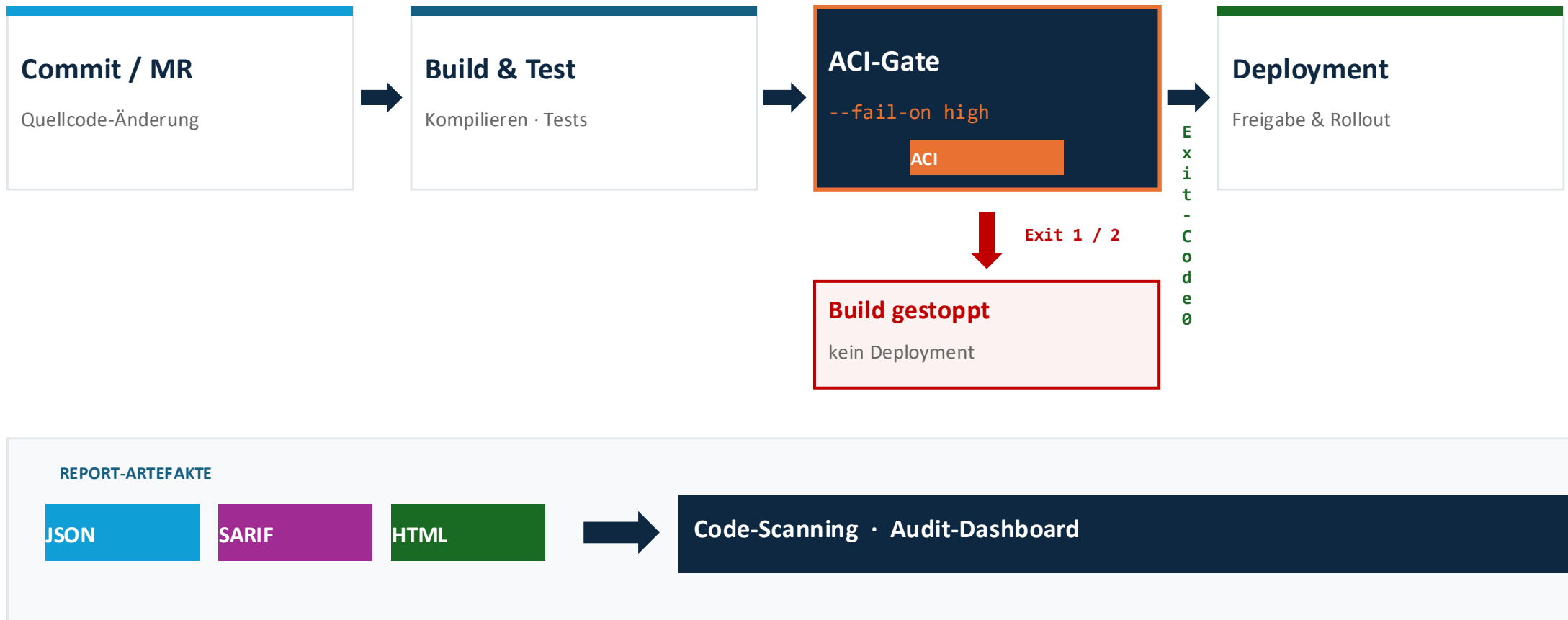


Schalter zur Absicherung

- **`--no-context`**
Kein Quelltext-Kontext in Reports — nur Fundstelle.
- **`--redact-secrets`**
Geheimnis-Muster maskiert, inkl. PostgreSQL/EPAS PASSWORD '...'.
`PGPASSWORD=secret psql -h host -U user -d database`
- **`--safe-report`**
Bündelt `--no-context` + `--redact-secrets` + `--redact-paths`. Empfohlen für CI-Artefakte.
- **`--context-lines N`**
Feinkorn-Kontrolle (0 = aus). Standard ist 3.

ACI als blockierender Gate in der Pipeline

Exit-Code 0 gibt frei · Exit-Code 1/2 stoppt den Build. Report-Artefakte fließen ins Code-Scanning.



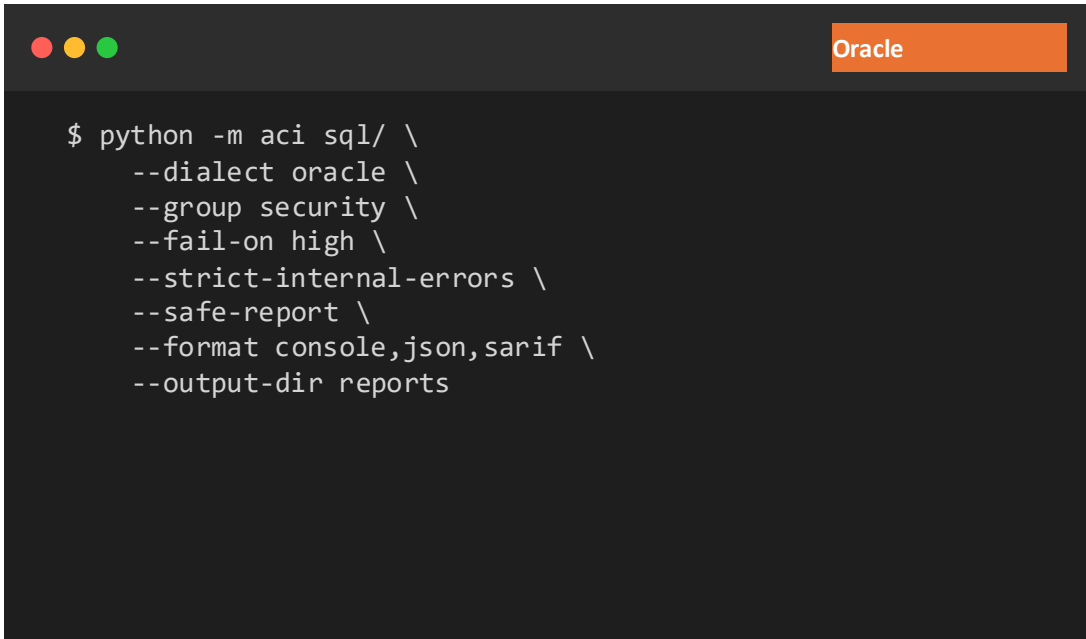
Exit-Codes — die drei Signale an die Pipeline

Der Exit-Code ist die einzige Information, die ACI nach außen gibt. Er bestimmt, ob der Build durchgeht.

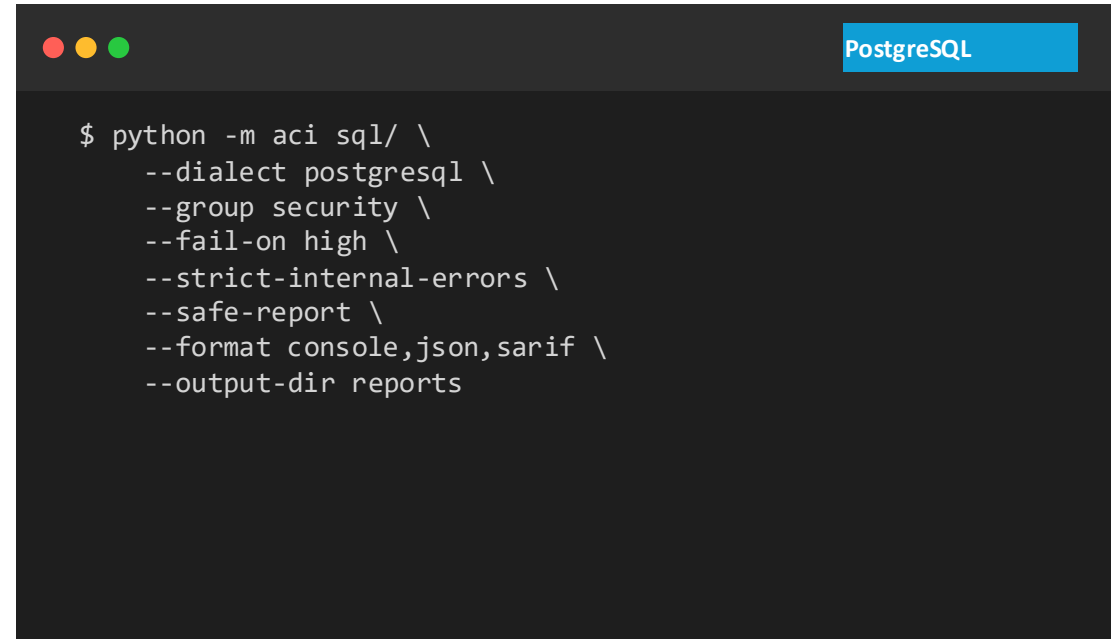
0 Exit	GATE BESTANDEN Kein Finding erreicht die Schwelle Build geht durch. Der Lauf war erfolgreich, alle Findings liegen unter der mit <code>--fail-on</code> gesetzten Schwelle (oder es gab keine Findings). ► Build fortsetzen → Deployment
1 Exit	FINDINGS ÜBER SCHWELLE Mindestens ein Finding erreicht / übersteigt die Schwelle Build stoppt. Ein blockierendes Finding (z.B. ein Critical bei <code>--fail-on critical</code>) wurde gemeldet. Das gewünschte Verhalten eines funktionierenden Gates. ► Findings beheben oder via Waiver dokumentieren
2 Exit	WERKZEUG- / EINGABEFEHLER Tool-Fehler oder durch <code>--strict-*</code> erzwungener Abbruch Build stoppt. Ungültige Regeldatei, Pfad nicht gefunden, ungültige Option — oder ein interner Check-Fehler mit <code>--strict-internal-errors</code> bzw. eine fehlerhafte Waiver-Datei mit <code>--strict-waivers</code>. ► Fehlermeldung auf <code>stderr</code> prüfen — kein stilles Durchwinken

Empfohlener Gate-Aufruf

Blockiert den Build bei sicherheitsrelevanten Findings und erzeugt zugleich archivierbare, kontextfreie Report-Artefakte.

A terminal window with a dark background and a title bar containing three colored dots (red, yellow, green) and the word "Oracle" in an orange box. The terminal shows a command to run a Python script for Oracle database security analysis.

```
$ python -m aci sql/ \
  --dialect oracle \
  --group security \
  --fail-on high \
  --strict-internal-errors \
  --safe-report \
  --format console,json,sarif \
  --output-dir reports
```

A terminal window with a dark background and a title bar containing three colored dots (red, yellow, green) and the word "PostgreSQL" in a blue box. The terminal shows a command to run a Python script for PostgreSQL database security analysis.

```
$ python -m aci sql/ \
  --dialect postgresql \
  --group security \
  --fail-on high \
  --strict-internal-errors \
  --safe-report \
  --format console,json,sarif \
  --output-dir reports
```

AUDIT-RELEVANT

Mindestens `--fail-on high` für ein verlässliches Gate

`--fail-on critical` ist zu schwach: ALTER SYSTEM SET, CREATE EXTENSION dblink und serverseitiger Dateizugriff sind oft mit High bewertet, nicht Critical.

Die Schalter im Einzelnen

Jeder Schalter hat einen klaren Zweck — gemeinsam ergeben sie einen funktionierenden, gehärteten Gate.

--group security

NUR SICHERHEITS-FINDINGS BLOCKIEREN

Coding-Guideline-Findings bleiben beratend, blockieren den Build nicht.

--fail-on high

SCHWELLE FÜR EXIT-CODE 1

Blockiert ab Schweregrad High — deckt also High und Critical ab.

--strict-internal-errors

STILLE PRÜFLÜCKEN VERHINDERN

Schlägt ein Check intern fehl, endet der Lauf mit Exit-Code 2 statt mit verstecktem Pass.

--safe-report

SICHERE ARTEFAKTE

Bündelt --no-context und --redact-secrets — Reports enthalten keinen Quelltext, keine Geheimnisse.

--format console,json,sarif

DREI AUSGABEKANÄLE

Konsole fürs Build-Log, JSON zur Auswertung, SARIF fürs Code-Scanning-Dashboard.

--output-dir reports

ZIELVERZEICHNIS

Hier landen die JSON-/SARIF-/HTML-Artefakte für die CI-Archivierung.

Waiver — der kontrollierte Ausnahmeprozess

Reale Pipelines brauchen einen Weg, einzelne Befunde zu akzeptieren — ohne den Gate als Ganzes zu schwächen.

PROBLEM

Einzelne akzeptierte Befunde — wie umgehen ohne `--fail-on` global abzuschwächen oder Regeln zu deaktivieren?

LÖSUNG: WAIVER

Versionierte JSON-Datei mit einem Eintrag je Ausnahme. `--waivers aci-waivers.json` — sichtbar im Report, aber außerhalb des `--fail-on-Gates`.

Eigenschaften eines Waivers



Sichtbar

Bleibt im Report — deutlich als »Waived« markiert mit Ticket, Owner, Ablaufdatum und Begründung.



Inhaltsgebunden

Über SHA-256-Fingerabdruck an den Code-Inhalt geknüpft — nicht an die Zeilennummer.



Befristet

Pflicht-Ablaufdatum erzwingt Wiedervorlage — keine ewigen Ausnahmen.



Auditierbar

Datei unter Versionskontrolle — Änderungen laufen über regulären Pull-Request-Review.

Waiver-Lebenszyklus — die vier Zustände

ACI prüft den Zustand jedes Waivers bei jedem Lauf und warnt aktiv über Wartungsbedarf.

 GÜLTIG	BEDINGUNG <code>Ablaufdatum ≥ heute + Fingerabdruck trifft ein Finding</code>	WIRKUNG Finding wird Waived: sichtbar im Report, zählt nicht für --fail-on. In SARIF als suppressions (kind: external).
 BALD FÄLLIG	BEDINGUNG <code>Noch gültig + Ablauf ≤ 14 Tage entfernt</code>	WIRKUNG Waiver greift weiter, aber ACI weist im Report auf die Wiedervorlage hin — rechtzeitig erneuern oder Code beheben.
 ABGELAUFEN	BEDINGUNG <code>Ablaufdatum < heute</code>	WIRKUNG Waiver greift nicht mehr — das Finding zählt wieder voll. ACI warnt aktiv über abgelaufene Waiver, der Build kann brechen.
 VERWAIST	BEDINGUNG <code>Fingerabdruck trifft kein Finding</code>	WIRKUNG ACI meldet den Waiver als entfernbar — der Code wurde geändert (Fingerprint passt nicht mehr) oder es lag ein Tippfehler vor.

Waiver-Datei — Aufbau & Verwendung

aci-waivers.json — eine JSON-Liste, jeder Eintrag ein Waiver. Über --waivers an ACI übergeben.

```
aci-waivers.json

[
  {
    "fingerprint": "39e7c9525757badd",
    "ticket": "SEC-1423",
    "owner": "ak@red-database-security.com",
    "expires": "2026-09-30",
    "reason": "Dyn. SQL nur über validierte\nWhitelist-Konstante;\nRefactor in SEC-1423 geplant.",
    "created": "2026-05-24",
    "risk_accepted": true
  }
]
```

Pflichtfelder

fingerprint	Aus dem Report kopieren
ticket	Issue-Tracker-Referenz
owner	Verantwortliche E-Mail
expires	YYYY-MM-DD
reason	Begründung der Ausnahme

Optional

created	Datum der Erstellung
risk_accepted	Bool — Risiko formell akzeptiert

VERWENDUNG IN DER PIPELINE

```
python -m aci sql/ \
  --waivers aci-waivers.json
```

Regelintegrität — Schutz vor manipulierten Regeln

Wer die Regeln verändert, kann den Gate schwächen. ACI macht den verwendeten Regelstand überprüfbar.

Ruleset-Hash

SHA-256

Stabiler, reproduzierbarer Hash über den Inhalt aller geladenen Regeldateien (Sicherheit, Guidelines, MITRE). Teams prüfen ihn in der Pipeline gegen einen erwarteten Wert.

Vertrauensstatus je Datei

trusted / untrusted

ACI weist je Regeldatei aus, ob sie zur gebündelten Installation gehört (trusted) oder aus einem benutzerdefinierten Pfad stammt (untrusted). Untrusted-Dateien werden sichtbar markiert und auf stderr gewarnt.

Wo der Hash erscheint

Konsole

Zeile »Regeln«

JSON

ruleset.integrity

HTML

Zeile »Regelintegrität«

SARIF

```
runs[ ].properties.aci_rule  
set_hash
```

HARTER KONTROLLPUNKT

--require-trusted-rules

ACI bricht mit Exit-Code 2 ab, sobald eine Regeldatei von außerhalb des Pakets geladen würde. Eine manipulierte Regel kann den Gate so nicht still schwächen.

SARIF — feinere Regelgruppierung

Der SARIF-Regelkatalog wird je konkreter Regelreferenz geführt — nicht nur je check_id.

VORHER · NUR check_id

Alle unerwünschten Packages laufen in einem Regel-Eintrag zusammen — Dashboards können einzelne Pakete nicht getrennt verfolgen.

```
"rules": [  
  { "id": "ACI-PKG", ... }  
]  
  
↳ utl_http, dbms_lob,  
   dbms_scheduler, ... alle hier
```

NACHHER · check_id:rule_ref

Jede Regelreferenz erhält ihren eigenen Katalog-Eintrag — GitHub/GitLab/Azure DevOps können jedes Paket oder jeden MITRE-Indikator getrennt verfolgen.

```
"rules": [  
  { "id": "ACI-PKG:utl_http", ... },  
  { "id": "ACI-PKG:dbms_lob", ... },  
  { "id": "ACI-OBf:CHR-CHAIN", ... },  
  { "id": "ACI-MITRE:T1059", ... }  
]
```

PARTIALFINGERPRINTS

Jeder Befund trägt seinen Fingerabdruck — direkt für Waiver verwendbar.

SUPPRESSIONS

Gewaiverte Findings erscheinen als suppressions (kind: external).

PROPERTIES

check_id und rule_ref stehen zusätzlich in den properties jedes Regeleintrags.

Plattform-Integration

ACI ist plattformneutral — Exit-Codes + SARIF sind universelle CI/CD-Schnittstellen.



GitHub Actions

Code Scanning

- 1 Job: ubuntu-latest, python 3.9+
- 2 `python -m aci sql/ --safe-report`
- 3 `upload-sarif@v3` → Security tab
- 4 Branch protection auf Exit-Code



GitLab CI/CD

SAST-Integration

- 1 Stage: test oder security
- 2 `image: python:3.9-slim`
- 3 `artifacts: paths: [reports/]`
- 4 `report:sast` → Security Dashboard



Azure DevOps

Pipelines + Scanning

- 1 Task: UsePythonVersion@0
- 2 `script: python -m aci sql/ ...`
- 3 `PublishCodeAnalysisLogs@1`
- 4 Branch Policy auf Build-Status

Gemeinsam: Exit-Codes treiben den Gate, SARIF speist die Security-Dashboards, Reports werden als Build-Artefakte archiviert.

GRENZEN

Was ACI nicht kann

ACI ist ein heuristischer Review-Assistent — kein vollständiges Code-Review und kein formaler Sicherheitsbeweis.

Bekannte Grenzen der Parser-/IR-Schicht



Kein vollständiger Oracle-PL/SQL- oder PostgreSQL-PL/pgSQL-Compiler



Keine vollständige Datenflussanalyse über Funktionsaufrufe hinweg



Begrenztes Verständnis komplex generierter SQL-Fragmente



Keine vollständige Kontrollflussanalyse



Keine semantische Validierung gegen eine laufende Datenbank



False Positives und False Negatives bleiben möglich

EMPFEHLUNG

ACI als ersten, automatisierten Filter in den Entwicklungs- und Freigabeprozess einbinden — die Findings als Ausgangspunkt für gezieltes manuelles Review verwenden.

Drei-Säulen-Ansatz für sichere Datenbank-Auslieferung

ACI

Automatischer Filter

+

Manuelles Review

Tiefe Prüfung

+

DB-Härtung

Laufzeit-Schutz

False Negatives bei dynamischem & generiertem SQL

ACI sieht nicht, was erst zur Build- oder Laufzeit entsteht. Vier typische Quellen unentdeckter Befunde.



Cross-File dynamisches SQL

Ein SQL-String wird in einer Datei oder Routine zusammengesetzt und in einer anderen ausgeführt. ACIs Datenflussanalyse ist routine-lokal und folgt Aufrufen nicht.



Build-Templates & Code-Generatoren

Wird der SQL-Code erst durch ein Template-/Makro-System, einen Präprozessor oder einen Generator erzeugt, prüft ACI nur die Vorlage, nicht das Ergebnis.



Migrations-Generatoren

Werkzeuge, die SQL erst zur Laufzeit aus einem Modell ableiten (ORM-Migrationen, Schema-Diff-Tools), liefern ACI keinen statisch prüfbaren Quelltext.



Konfigurations- & datengetriebenes SQL

SQL-Fragmente aus Konfigurationsdateien, Datenbankinhalten oder externen Eingaben, die erst zur Laufzeit zusammengesetzt werden, liegen außerhalb des analysierten Textes.

Warum ACI keinen vollständigen Compiler hat

Bewusste Architekturentscheidung — der Aufwand stünde in keinem Verhältnis zum Nutzen.

KERNAUSSAGE

Für reale Datenbankbestände — 99 % DDL (Tabellen, Views, Sequences, Grants), 1 % prozeduraler Code — würde ein vollständiger Compiler die Qualität kaum verbessern.

PROZEDURALER CODE

~1 %

Hier braucht es echte Tiefe

- Datenfluss & Kontrollfluss
- Sichtbarkeitsbereiche, Typauflösung
- Ausdrucksauswertung über Routinegrenzen
- Dynamisches SQL, Taint-Quellen

→ ACIs IR-, Taint- und SQLI-Maschinerie bedient genau diese Fälle.

DDL (TABELLEN, VIEWS, GRANTS)

~99 %

Hier reichen Lexer + Regeln

- Deklarative Einzelanweisungen
- Kein Datenfluss, kein Kontrollfluss
- Statement-Abgrenzung + Mustererkennung
- Qualität skaliert mit Regelbreite, nicht Parser-Tiefe

→ ACI hat heute beides: korrekte Statement-Grenzen und gute Regeln.

Drei Stufen, nicht zwei

Sauber getrennt: Es gibt nicht nur Parser ja/nein, sondern drei deutlich unterschiedliche Investitionsstufen.

HEUTE

1

Lexer + Regeln

Aktueller ACI-Stand für DDL

- Erkennt Schlüsselworte und Statement-Grenzen
- Mustererkennung über JSON-Regeln
- Reicht für CREATE TABLE/VIEW/SEQUENCE und GRANT in den allermeisten Fällen

Aufwand: gering · Inkrementell

OPTIONAL

2

Echte Grammatik / AST

Strukturierte Sicht — kein Compiler

- Beseitigt die letzten Statement-Abgrenzungs-Kanten
- Für komplexe DDL: verschachtelte Constraints, Partitionierung
- Für PL/SQL: würde FP/FN bei SQLi und dynamischem SQL reduzieren

Aufwand: hoch · Bricht Null-Abhängigkeit

NICHT ZIELFÜHREND

3

Vollständiger Compiler

Semantik & Symbolauflösung

- Beantwortet: existiert diese Spalte, stimmt der Typ?
- Funktioniert offline nicht — braucht den DB-Katalog
- ACI ist bewusst statisch und DB-los: Semantik ohne Schema bringt der Compiler nicht

Aufwand: prohibitiv · Architekturbruch

Der Hebel liegt im DDL-Regelsatz

Qualität bei DDL-lastigen Beständen kommt aus Regelbreite — nicht aus Parser-Tiefe.

WAS DIE 99 % QUALITÄT WIRKLICH BRINGT

Tiefe des DDL-Regelsatzes

- Riskante Grants und PUBLIC-Synonyme/-Privilegien
- Fehlende Constraints, gefährliche Defaults
- Tablespace-/Verschlüsselungs-Policy
- WITH CHECK OPTION bei Views, Sequence-Rechte
- ALTER-Operationen und Privilegienänderungen

EHRliche BEOBACHTUNG

ACIs aufwendigste Maschinerie — IR, Routine-Grenzen, Taint, SQLi — bedient faktisch die 1 %.

Für DDL-lastige Bestände hat weiterer Aufwand auf der prozeduralen Seite geringen Hebel.

Der Hebel liegt bei DDL-Regeltiefe.

EMPFOHLENE REIHENFOLGE

1 DDL-Regeln verbreitern
billig, inkrementell

2 Kunden-DDL-Korpus prüfen
reale Lücken finden

3 Optional: kleine DDL-Grammatik
nur CREATE TABLE/VIEW/SEQ/GRANT

Werkzeuge zur Fehleranalyse

Drei Schalter helfen, einen ACI-Lauf im Fehlerfall einzugrenzen — vom Tool-Crash bis zum fragwürdigen Finding.

--debug

Voller Python-Traceback

Zeigt bei einem unerwarteten Fehler den vollständigen Python-Traceback statt nur einer Kurzmeldung. Erste Wahl bei Tool-Crashes oder unklaren Fehlermeldungen.

WANN VERWENDEN

ACI bricht mit unklarer Meldung ab

--dump-ir

IR-Sicht einer Datei

Gibt die Parser-/IR-Sicht einer einzelnen Datei als JSON aus und beendet sich. Hier sieht man, wie ACI Routinen, Statements und dynamisches SQL erkannt hat.

WANN VERWENDEN

Fragwürdiges Finding — Statement falsch abgegrenzt?

--list-checks

Aktive Regel-Abdeckung

Zeigt, welche Checks und Regeln für Dialekt und Gruppe aktiv sind. Fehlt ein erwartetes Finding, ist oft die zugehörige Regel inaktiv oder der Dialekt falsch gesetzt.

WANN VERWENDEN

Erwartetes Finding wird nicht gemeldet

INTERNE FEHLER • ACI-INTERNAL • HIGH

Schlägt ein Check intern fehl, stoppt das den Scan nicht. Der Fehler erscheint als Finding der Gruppe »Interner Fehler« — ein Werkzeugfehler, kein Code-Mangel. Mit `--debug` reproduzieren, mit `--dump-ir` der betroffenen Datei eingrenzen.

Workflow im Fehlerfall

Vier Schritte, ein Lauf eingegrenzt — von der Werkzeugmeldung bis zur reproduzierbaren Ursache.



PRAGMATISCHER WEG

Lokale Änderungen mit »python3 -m compileall -q aci« auf Syntaxfehler prüfen. Akzeptierter Einzelfall: Waiver dokumentieren statt Gate abschwächen. Im Zweifel ein gezieltes manuelles Review.

Workflow im Fehlerfall

Vier Schritte, ein Lauf eingegrenzt — von der Werkzeugmeldung bis zur reproduzierbaren Ursache.

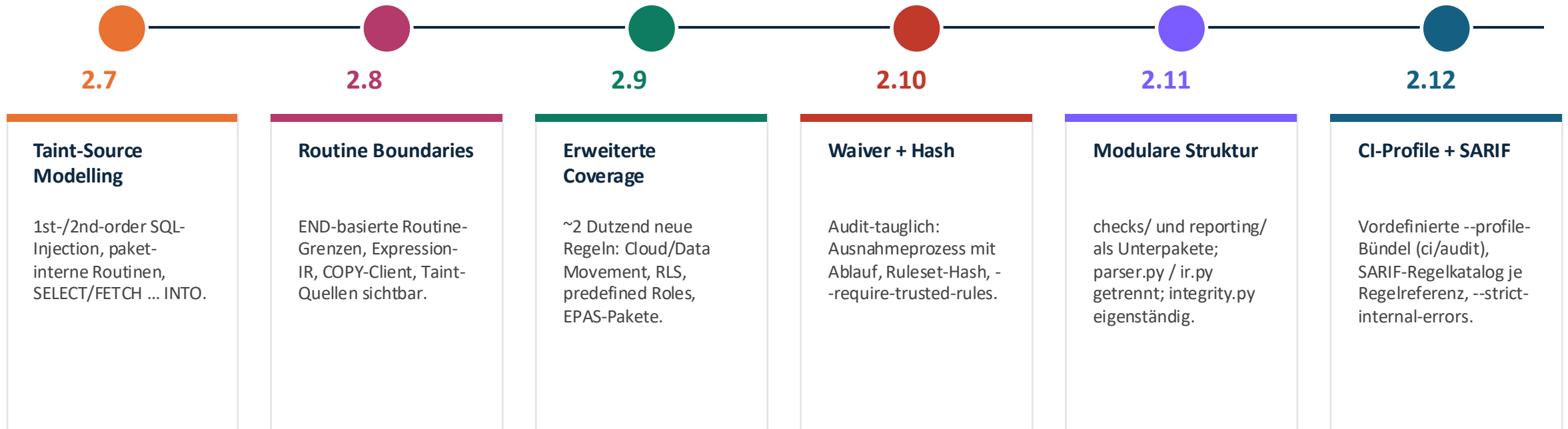


PRAGMATISCHER WEG

Lokale Änderungen mit »python3 -m compileall -q aci« auf Syntaxfehler prüfen. Akzeptierter Einzelfall: Waiver dokumentieren statt Gate abschwächen. Im Zweifel ein gezieltes manuelles Review.

Versionshistorie 2.7 → 2.20

Wie sich ACI von einem reinen Pattern-Scanner zum strukturierten Review-Werkzeug mit CI/CD-Härtung entwickelt hat.

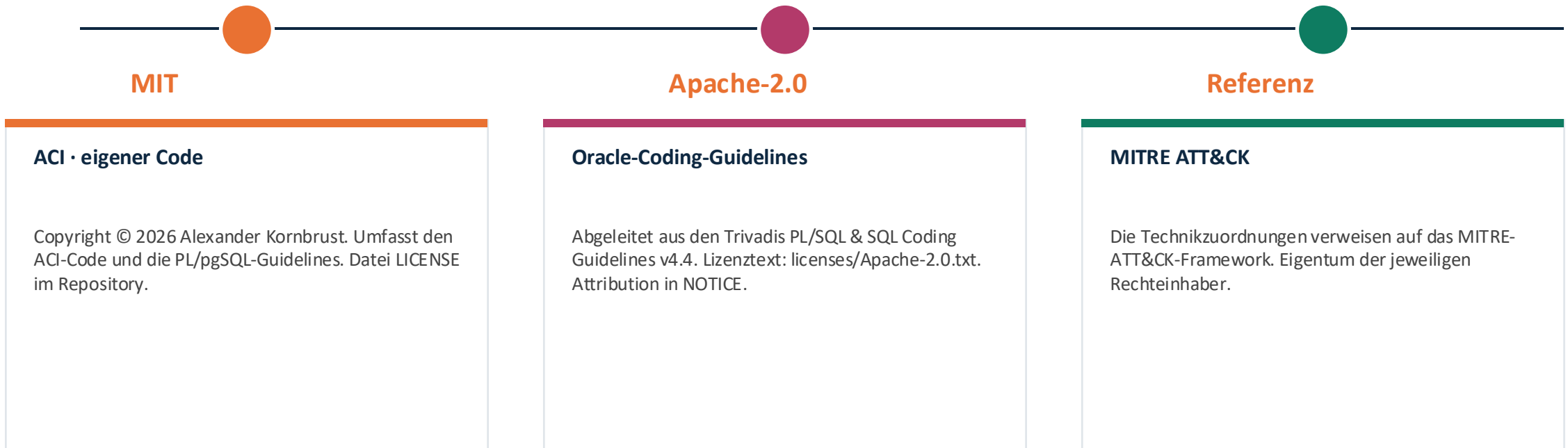


AKTUELL · 2.20.1

ACI 2.20.1 erweitert Sicherheit & Hardening deutlich: vordefinierte CI-Profile, kontrollierter Waiver-Ausnahmeprozess, überprüfbarer Regelstand (Ruleset-Hash) und feingranulare SARIF-Integration für Code-Scanning-Dashboards.

Open Source · gemischt lizenziert

ACI ist Open Source. Der eigene Code steht unter MIT; abgeleitete Regelinhalte tragen ihre Originallizenz.



HINWEIS

Regeldateien sind Teil der Trust Boundary — wer aci/rules/ manipulieren kann, schwächt den CI/CD-Gate. ACI weist je Datei den Vertrauensstatus aus; mit --require-trusted-rules wird daraus ein harter Kontrollpunkt.

Glossar — Begriffe auf einen Blick

Die wichtigsten Fachbegriffe aus dieser Präsentation und der ACI-Dokumentation.

Check

Prüfeinheit im Code, die eine oder mehrere Regeln auswertet.

Finding

Einzelner Befund mit Regel, Schweregrad, Datei, Zeile, Meldung.

Maskierung

Codefassung, in der Kommentar- und String-Inhalte ersetzt sind.

Sink

Stelle, an der ein getainteter Wert gefährlich wird (EXECUTE IMMEDIATE).

Waiver

Kontrollierte, befristete Ausnahme mit Ticket, Owner, Begründung.

SARIF

Static-Analysis-Format für GitHub/GitLab/Azure-Code-Scanning.

Regel

In JSON beschriebenes Prüfmuster — Wortliste, Regex oder Built-in.

IR

Leichtgewichtige Zwischenform (Routinen, Statements, dyn. SQL).

Taint

Wert aus einer ungeprüften Quelle (Parameter, Tabelle, Cursor).

Fingerprint

SHA-256-Hash, der einen Waiver inhaltsgebunden an ein Finding bindet.

Ruleset-Hash

SHA-256 über alle geladenen Regeldateien — macht Regelstand prüfbar.

MITRE ATT&CK

Wissensbasis für Angreifer-Taktiken — ACI ordnet Muster Techniken zu.

Vielen Dank.



Fragen und Diskussion

Sicherheit

Coding Guidelines

CI/CD-Gate